

Le Classi in Java

Una classe in Java è un modello o uno stampo per creare oggetti. È il fondamento della programmazione orientata agli oggetti (OOP), in cui si modellano concetti del mondo reale come entità con proprietà e comportamenti.

Struttura e Funzione

Una classe definisce attributi (variabili che rappresentano le caratteristiche dell'oggetto) e metodi (funzioni che rappresentano il comportamento).

Esempio:

```
public class Persona {  
    String nome; // attributo  
    int età;    // attributo  
  
    void saluta() { // metodo  
        System.out.println("Ciao, sono " + nome);  
    }  
}
```

Oggetti e Istanze

Gli oggetti sono istanze di una classe, creati con l'operatore new.

Ogni oggetto ha i propri valori per gli attributi, ma condivide i metodi definiti nella classe.

```
Persona persona1 = new Persona();  
persona1.nome = "Alice";  
persona1.saluta(); // Output: Ciao, sono Alice
```

Caratteristiche Chiave

- **Totale orientamento agli oggetti:** Oltre ai tipi primitivi (int, float, ecc.), tutto in Java è una classe o un oggetto.
- **Incapsulamento:** I dati (attributi) e le operazioni (metodi) sono racchiusi in una singola unità.
- **Modificatori di accesso:** Come **public**, **private**, **protected**, controllano chi può accedere ai membri della classe.
- **Astrazione:** Fornisce un'interfaccia semplificata nascondendo la complessità.
- **Polimorfismo:** Consente a oggetti di classi diverse di essere trattati come istanze di una classe comune.
- **Ereditarietà:** (Spiegato più avanti) Permette a una classe di ereditare attributi e metodi da un'altra classe.

Importanza

Le classi permettono di organizzare il codice in modo modulare, leggibile e manutenibile, fondamentale per lo sviluppo di applicazioni complesse in Java.

NOTA BENE: In Java, ogni classe pubblica (**public class**) deve essere definita in un file **.java** con lo stesso nome della classe. Ad esempio, la classe `Persona` deve essere salvata in un file chiamato `Persona.java`

Il metodo main

In Java, non esiste una "classe main" come concetto specifico, ma si fa riferimento comunemente alla classe che contiene il metodo main, che rappresenta il punto di ingresso di un'applicazione Java.

Il metodo main è l'entry point dell'applicazione: quando si esegue un programma Java, la Java Virtual Machine (JVM) cerca e invoca automaticamente questo metodo.

La sua sintassi è fissa:

```
public static void main(String[] args)
```

public: Accessibile da qualsiasi parte, inclusa la JVM.

static: Può essere chiamato senza creare un'istanza della classe.

void: Non restituisce alcun valore.

main: Nome convenzionale riconosciuto dalla JVM.

String[] args: Array che contiene gli argomenti passati da riga di comando.

Posizione del Metodo main

Può trovarsi in qualsiasi classe pubblica del programma. Tuttavia, per chiarezza, è buona pratica definirlo in una classe dedicata, spesso chiamata Main o con un nome significativo.

Esempio:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Avvio del programma!");  
    }  
}
```

Esecuzione

Il programma si avvia con:

```
java Main
```

Se vengono passati argomenti:

```
java Main arg1 arg2
```

questi saranno disponibili nell'array *args*

Il metodo main ha l'attributo *static* perché deve essere chiamato dalla JVM prima che venga creata qualsiasi istanza della classe.

Poiché all'avvio del programma non esistono ancora oggetti, il metodo main deve essere accessibile senza dover istanziare la classe. L'uso di *static* permette alla JVM di invocare *main()* direttamente tramite il nome della classe, senza bisogno di creare un oggetto.

Sintassi di una nuova istanza di un oggetto

Prendiamo il comando

```
Persona persona1 = new Persona();
```

Questa riga di codice in Java crea un nuovo oggetto (istanza) della classe Persona.

La sintassi significa:

Persona : Il tipo di dato, corrispondente al nome della classe. Indica che l'oggetto creato sarà un'istanza di Persona.

persona1 : Il nome della variabile oggetto, che conterrà il riferimento all'istanza creata.

= : Operatore di assegnazione.

new : Parola chiave che alloca memoria per un nuovo oggetto e invoca il costruttore della classe.

Persona() : Chiamata al costruttore della classe Persona. Se non è definito esplicitamente, viene usato il costruttore di default (senza parametri). Il costruttore è un metodo base di ogni classe che verrà spiegato più avanti.

Accesso ai metodi e agli attributi di un oggetto

Per poter accedere agli attributi e ai metodi di un oggetto c'è bisogno innanzitutto che l'oggetto sia stato istanziato (banalmente utilizzando new). L'accesso avviene quella che viene definita *dot-notation* (notazione puntata) come nell'esempio sopra (che qui riporto):

```
persona1.nome = "Alice"; //accesso a un attributo  
persona1.saluta();       //accesso a un metodo
```

Si può usare solo con membri pubblici (*public*), a meno che l'accesso avvenga all'interno della stessa classe

(Spiegato più avanti) Se un metodo restituisce un oggetto, si può applicare la dot notation a cascata: *oggetto.metodo().altroMetodo()*.

Lo Standard Output (*stdout*)

Lo standard output in Java è il canale predefinito per l'output di un programma, generalmente associato alla console o al terminale.

Viene accessibile tramite *System.out* e permette di stampare dati con metodi come:

System.out.print() - stampa senza andare a capo.

System.out.println() - stampa una linea (nel senso di stringa) e va a capo.

System.out.printf() - stampa con formattazione.

Di default, l'output appare sullo schermo (monitor/terminale), ma può essere reindirizzato a un file o a un altro processo, come una stampante.

Anche gli attributi possono essere *static*

Una variabile statica (o "di classe") è condivisa da tutte le istanze della classe: esiste un'unica copia di quella variabile, indipendente dal numero di oggetti creati.

Caratteristiche

Si dichiara con la parola chiave *static*.

Appartiene alla classe, non a un oggetto specifico.

Si accede tramite il nome della classe: `NomeClasse.variabile`.

Viene inizializzata una sola volta, al caricamento della classe.

Esempio

```
public class Persona {  
    String nome;  
    static int numeroIstanze = 0; // Condivisa da tutti  
  
    public Persona() {  
        numeroIstanze++; // Incrementa il contatore  
    }  
}
```

Uso:

```
Persona p1 = new Persona();  
Persona p2 = new Persona();  
System.out.println(Persona.numeroIstanze); // Stampa: 2
```

Principali problemi tra versioni differenti di Java

I principali problemi tra le versioni di Java riguardano la compatibilità e possono causare errori durante la compilazione o l'esecuzione.

Retrocompatibilità

Java è retrocompatibile nella maggior parte dei casi: un programma compilato con una versione vecchia può essere eseguito su una JVM più recente.

Tuttavia, non vale il contrario: se un programma è compilato con Java 11 e si tenta di eseguirlo su Java 8, si ottiene un errore:

```
java.lang.UnsupportedClassVersionError: ... compiled by a more recent version of the Java Runtime
```

Problemi comuni

- API non disponibili: Anche se si usa `-target 8` per compilare, si potrebbero usare metodi introdotti in versioni successive (es. `Set.of()` da Java 9). Il codice compila ma fallisce a *runtime* con `NoSuchMethodError`.
- Cambiamenti nel linguaggio: Nuove parole chiave o sintassi (es. `var` da Java 10) non sono riconosciute dalle vecchie versioni del compilatore.
- Deprecazioni e rimozioni: Alcune classi o metodi vengono deprecati e poi rimossi (es. `Applet`), rompendo codice esistente.

Soluzioni

Usa i flag `-source` e `-target` per specificare la versione minima.

Meglio ancora, usa `-release` (da Java 9) che garantisce compatibilità sia linguistica che API.

Assicurati che la JRE di esecuzione sia uguale o superiore alla versione di compilazione.

Ad esempio in Java 25 tra le tante funzionalità sono stati introdotti

Compact Source Files e **Instance Main Methods** (JEP 512): Permette di scrivere programmi Java semplici senza boilerplate:

```
void main() {  
    println("Hello, World!");  
}
```

Eseguibile direttamente con `java hello.java`.

IDE consigliati

Un IDE (Integrated Development Environment, o ambiente di sviluppo integrato) è un software che unisce in un'unica interfaccia strumenti essenziali per lo sviluppo di programmi, come:

Editor di codice con evidenziazione della sintassi e completamento automatico.

Compilatore o interprete per tradurre ed eseguire il codice.

Debugger per individuare e correggere errori.

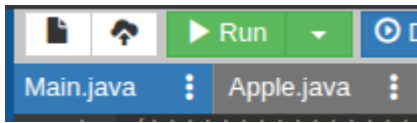
Strumenti di automazione (build, test, deployment) e supporto a sistemi di controllo versione.

L'IDE semplifica e velocizza la scrittura, verifica ed esecuzione del codice, riducendo la necessità di passare tra programmi diversi

Gli IDE che si usano principalmente sono:

- **Eclipse**: progetto open source, maggiormente consigliato per lo sviluppo in ambiente Java;
- **IntelliJ**: progetto proprietario, che ha bisogno di un acquisto della licenza per accedere a tutte le funzionalità più professionali;
- **visual studio code**: è un editor che si avvicina a più linguaggi di programmazione (un po' come www.onlinegdb.com);
- **netbeans**: sviluppato inizialmente per java, attualmente, è un IDE multi-linguaggio scritto in Java.

Per quanto riguarda il codice scritto oggi in classe su www.onlinegdb.com, bisogna aprire una ulteriore scheda, tramite l'icona New File in alto a sinistra e nominarla *Apple.java* (trovandosi nella condizione)



a questo punto bisogna copiare nella scheda Main.java il seguente codice

```
public class Main
{
    public static void main (String[] args) {
        Apple apple = new Apple ();
        int volume = apple.volume;

        System.out.println ("La mela ha un volume pari a " + volume);
        apple.grow (10);
        System.out.println ("La mela ha un volume pari a " + volume);
        /* non ha stampato la modifica perché la variabile volume non coincide
           con l'attributo dell'oggetto apple */
        volume = apple.volume;
        System.out.println ("La mela ha un volume pari a " + volume);

        String appleMessage = apple.fall();
        System.out.println (appleMessage);
    }
}
```

mentre bisogna incollare il seguente codice nella scheda Apple.java:

```
public class Apple {
    int volume = 0;
    String color = "rosso";

    public void grow (int increment) {
        volume += increment;
    }

    public String fall () {
        return "Cadooo!";
    }
}
```